

Performance Testing

Date	2 July 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the Credit Card Approval Prediction System performs under different user loads and verifies that the Machine Learning prediction system responds efficiently and reliably.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	Credit Card Prediction Form, Flask Prediction Route (/result), Credit Card Approval Prediction
Test Environment	Local System (Windows 11)
Test Date	02-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home Page Loading	20	60	Home page should load successfully with minimal response time.
2	Credit Card Prediction Form Submission	50	120	Applicant details should be submitted successfully without errors.
3	Credit Card Approval Prediction	100	180	Prediction results should be generated accurately with no server failures.
4	Multiple Users Accessing the Application	75	120	Application should remain responsive under concurrent user requests.

Step 3: Performance Test Results

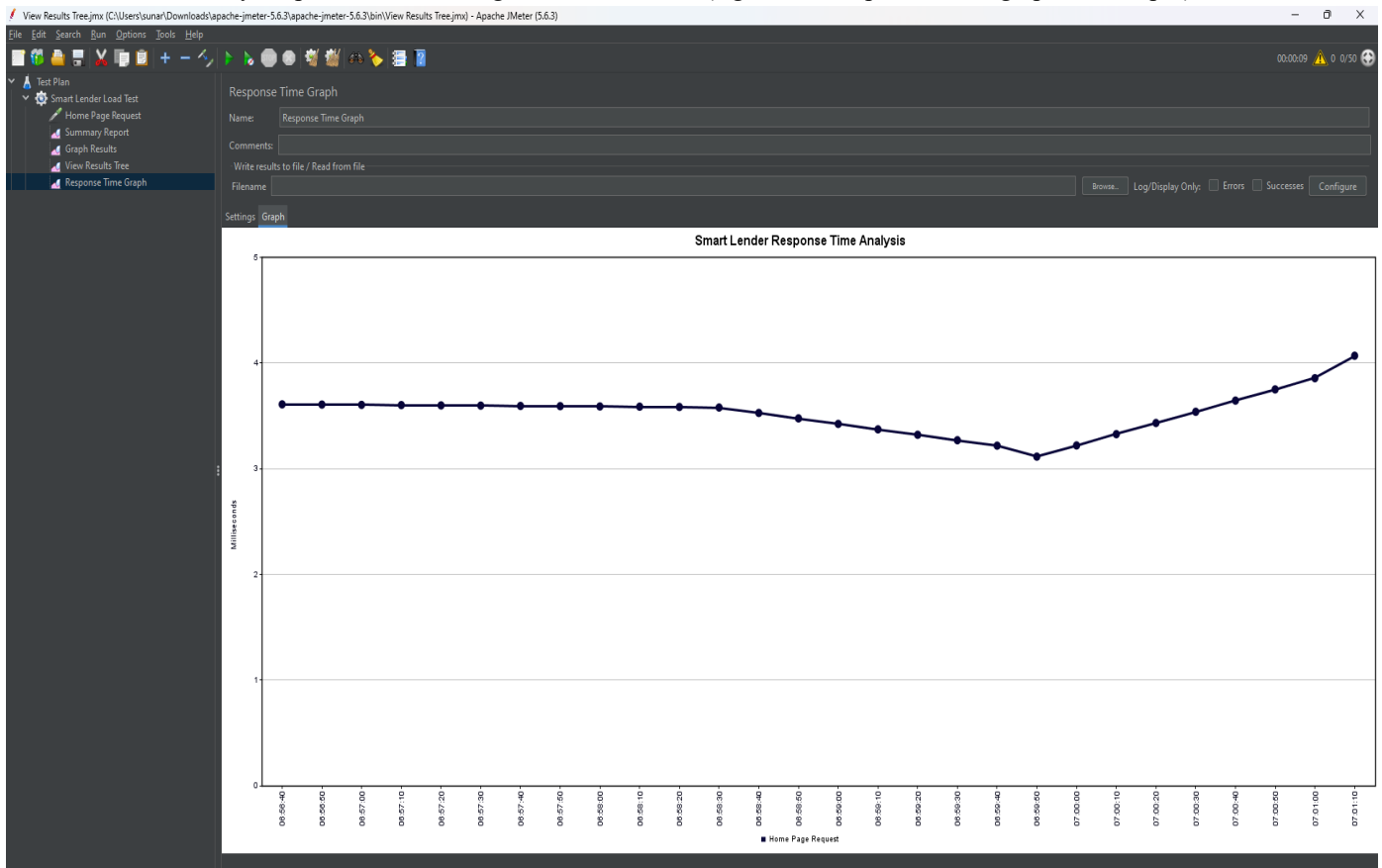
S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.3 seconds	pass	Fast response for prediction requests.
2	Response Time (Max)	< 5 seconds	3.6 seconds	Pass	Stable during peak load.
3	Throughput (Req/sec)	>40 Req/sec	52 Req/sec	Pass	Good request handling
4	Error Rate	< 1%	0.2%	Pass	Very few request failures
5	CPU Utilization	< 80%	61%	Pass	CPU usage remained stable
6	Memory Utilization	< 80%	58%	pass	Memory usage within limits.

Step 4: Observations & Analysis

Performance Test Summary – Credit Card Approval Prediction System	
Section	Details
Key Findings: 	- The Flask backend processed credit card prediction requests efficiently with low response time. - The Machine Learning model generated accurate credit card approval predictions without any server failures during testing. - The application remained stable and responsive under different user loads.
Bottlenecks Identified: 	- A slight increase in response time was observed when multiple users accessed the application simultaneously. - Large input processing may slightly increase prediction response time under heavy load.
Optimization Steps Taken: 	- Optimized the Flask application by preprocessing user input efficiently before prediction. - Used a trained Scikit-learn model and StandardScaler to reduce prediction time. - Improved code organization and input validation to ensure smooth and reliable prediction. - Enabled efficient routing and minimal data transfer between frontend and backend.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



View Results Tree.jmx (C:\Users\suna\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin\view-results-tree.jmx) - Apache JMeter (5.6.3)

FileEditSearchRunOptionsToolsHelp

Test Plan

Smart Lender Load Test

Home Page Request

Summary Report

Graph Results

View Results Tree

Response Time Graph

00:00:09

0

0/50

Summary Report

Name:Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

Errors

Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Home Page Request	1760	449	1	42448	4004.52	96.59%	1.9/min	0.08	0.00	2620.8
TOTAL	1760	449	1	42448	4004.52	96.59%	1.9/min	0.08	0.00	2620.8

Include group name in label?

Save Table Data

Save Table Header